

Bevezetés a Rust programozásba

Gembela Gergely

1. óra: Követelmények, környezet

<https://github.com/bmeaut/VIAUBXAV090>

A tárgy oktatói

- Tárgyfelelős: Somogyi Ferenc Attila (Somogyi.Ferenc@aut.bme.hu)
- **Gembela Gergely** (gergelygembela@edu.bme.hu, QB234)
 - Az előadásokkal kapcsolatban visszajelzést, kérést, javaslatokat nekem érdemes küldeni
- Mezei Gergely (gmezei@aut.bme.hu)
- Teamsen is elértek minket

Követelmények

- Géptermi ZH a 14. heti órán (Rust feladatok, fordító+linter használható)

```
let jegy = match pontszám {  
    0..50 => elégtelen,  
    50..60 => elégséges,  
    60..75 => közepes,  
    75..85 => jó,  
    85..=100 => jeles,  
    _ => nem_teljesítette  
};
```

- Pót ZH a póthéten

Jegy szerzése előadással

- A következő témákban várunk 25-30 perces előadásokat:
 - Grafikai fejlesztés Rust-ban (pl. Vulkan API felett)
 - Rust a Linux kernelben
 - Rust a Windows ökoszisztémában (beleértve a Rust Winrt-t)
 - Rust interop más nyelvekkel, pl. Python, C#, Js (projekt+bemutató)
 - Akár natívan (pl. PyO3)
 - Akár pl. WebAssembly target használatával

Jegyzetek, a tárgy anyagai

- A tárgy feltételezi, hogy tudtok C++-ban programozni, így lesz ahol összehasonlításra/különbségekre építünk
- Minden anyag elérhető a tárgy GitHub oldalán
- Javításokat, javaslatokat PR-ben várunk
 - A jegyzet egyelőre Markdown, később Zensical alapú lesz
 - A diák Marp-ban készülnek, elképzelhető hogy a forrást is elérhetővé tesszük

A félév menete

- A félév első felében megismerjük a Rust nyelv alapvető elemeit
 - az első 5-7 előadás általános nyelvi ismereteket ad, bemutatja a borrow checker működését, és a Rust típusrendszerét
 - foglalkozunk a standard könyvtár elemeivel, best-practice hibakezeléssel, metaprogramozással, generikusokkal, és életcikluskezeléssel, tárolókkal, smart pointerekkel
 - igyekszünk bemutatni, hogy bár a nyelv sok ponton próbálja megakadályozni, van lehetőségünk *lábon löni* magunkat

A félév menete

- A félév második felében szerepet kap a többszálú fejlesztés, a kód terjesztése (pl. könyvtárként), interoperabilitás más nyelvekkel
- A fennmaradó előadásokon különböző gyakorlati példákkal foglalkozunk, valamint sor kerül a hallgatói előadásokra
- A félévet a ZH zárja (14. héten, az előadás idejében)
 - A ZH alternatívája a saját előadás kidolgozása és megtartása, ezek az utolsó 4-5 hét előadásai végére lesznek ütemezve

A tárgy céljai

- Alapvető bevezetés a Rust programozásba
- Az erőforrások bemutatása, amik egyszerűvé teszik a tájékozódást a Rust ökoszisztémában
- Nem célja:
 - Haladó Rust szerkezeteket/koncepciókat számonkérni
 - Átfogó ismereteket adni beágyazott/no-std fejlesztéshez
 - Lefedni a Rust fejlesztés minden területét

Miért jó Rustban fejleszteni?

- Remek ökoszisztéma (build rendszer, könyvtárak, tesztelés, stb.)
- Garantálható a memóriakezelés biztonsága (akár több szálon is)
- Rengeteg hasznos könyvtár elérhető és egyszerűen felhasználható
- Jó a dokumentáció, a fordító hibaüzenetei is sokat segítenek
- Remek eszközkészlet más nyelvekkel való interoperabilitásra
- A felkapottsága miatt aktív közösség segíti egymást
- A standard könyvtár erős, kényelmessé teszi a nyelvet

Mit nem old meg a Rust fordító?

- Egyre elterjedtebb gyakorlat kódbázisokat Rust-ra portolni/fordítani
 - A Rust használata magában nem garancia a biztonságra
 - Rustban is lehet logikailag rossz/sebezhető kódot írni
- A Rust natív (gépi) kódra fordul
 - Nem triviális hogy ettől az eredmény *gyors* és *hatékony* is lesz
 - Nincs stabil Application Binary Interface (ABI)
- A fordítás *nem gyors*

A Rust fejlődése, verziói

- Új funkciók kiadásoktól függetlenül is bekerülnek
- Kb. 3 évenként új kiadás (2015, 2018, 2021, 2024)
- A projekt leírója tartalmazza a célzott kiadást
 - Ez a kompatibilitást segíti, hasonló a C++11, C++14, C++17, stb. kiadásokhoz, de a Rust nem törekszik verziókompatibilitásra
 - Emellett a nyelv és az eszközök is rendelkeznek verziószámmal
- Létezik **stable** és **nightly** csatorna, mi a **stable**-t használjuk

Polonius és új trait resolver

- A polonius egy új borrow checker, ami több elméleti szempontból helyes kódot fogad majd el
 - Nincs messze a kiadása, de ez már jó ideje így van :D
 - [Nightly release-ben már használható](#)
- Az [új trait solver](#) már elérhető nightly csatornán

Rust eszközök

- Eszközök kezelésére: [rustup](#)
- Fordító keretrendszer: `cargo`
 - Konkrét fordító: `rustc`
 - Formázáshoz: `Rustfmt`
- `rust analyzer` IDE support (LSP felett)
- [Rust Clippy](#).

Egyéb hasznos eszközök

- [RustViz](#): a későbbiekben élettartam vizualizációhoz
- [RustOwl](#): VS Code-hoz a borrow checker működésének ábrázolásához
- [Compiler Explorer](#): a fordítás elemzéséhez

Fejlesztői környezetek (IDE-k) és könyvtárkezelés

- **JetBrains RustRover**
- VS Code (+rust-analyzer extension)
- crates.io
- docs.rs

Hasznos anyagok, források

- [The Rust Book](#)
- [Rust by Example](#)
- [The Rust Reference](#)
- [Learning Rust With Entirely Too Many Linked Lists](#)
- [The Cargo Book](#)
- [The Rustonomicon](#)
- Let's get Rusty YouTube csatorna

Platformfüggetlenség

- A tárgyat első sorban a laborgépekre, így Windowsra terveztük
- Minden felhasznált technológia elérhető Linuxon és MacOS-en is
 - A platformspecifikus esetekben hasznos kondicionális fordítást későbbi laboron be is mutatjuk
- A toolchainek potenciális különbségei ebben az évben még nem kerültek felfedezésre/dokumentálásra, ha eltérést/problémát tapasztaltok, írjatok!

W1T0: Ez itt az első feladat dia

- A diasorokban a laborfeladatokhoz kapcsolódó részek ilyen stílussal jelennek majd meg
- Ha az előadás anyagát már ismeritek, vagy a bevezetővel párhuzamosan már dolgoznátok, keressétek a kék diákat
- A kék diák tartalma elérhető lesz a tárgy GitHub oldalán is, jellemzően kóddal/kiinduló projekttel kiegészítve

W1T0.1: A szükséges eszközök telepítése

- Telepítsük a Rust toolchaint, a `rustup-init` segítségével (letöltés+futtatás)
 - A telepítéskor megfelelőek az alapértelmezett beállítások, viszont ezek MSVC telepítést is igényelnek (a laborban ez adott)
 - Próbáljuk ki a telepítést a `rustc --version` parancs futtatásával
- Telepítsük a `rust-analyzer` VS Code kiegészítőt
 - A laborban ez már telepítve van

W1T1: Ismerkedés a `cargo`-val

- A Rust csomagkezelője és build rendszere
 - A csomagkezeléssel részletesebben később foglalkozunk
- Hozzunk létre egy új projektet: `cargo new <name>`
 - Ez alapértelmezetten egy `binary` projekt létrehozásához vezet
- Futtassuk a létrehozott projektet: `cargo run`
- Ha csak fordítani szeretnénk, a `cargo build` paranccsal megtehetjük
- Ha vannak tesztjeink, a `cargo test` paranccsal futtathatjuk őket

A program/crate struktúrája

- A crate-ben létrejön egy TOML leíró, és az `src/main.rs` fájl, a crate gyökér eleme
- Ha programot (tehát nem könyvtárat) eredményező crate-et készítettünk, szükséges egy `main` függvény (belépési pont)
 - Nincs visszatérési értéke, saját kilépési kódot pl. az `exit(42)` hívásával érhetünk el
 - Léteznek alacsonyabb szintű belépési pontot kezelő elemek, ezek jelenleg túlmutatnak az előadás célján, de [itt elérhetők](#)

Egyszerű konzolos IO

- kiírni a `print!()` és `println!()` függvényszerű makrókkal tudunk
 - Ezek a makrók fordításidőben generálnak kódot, és képesek formázott string feldolgozására (pl.

```
print!("Az eredmény: {}", az_eredmeny);
```

- Beolvasásra: [text_io](#), vagy

```
let mut result = String::new();  
io::stdin::().read_line(&mut result).expect("Could not read input!");
```

W1T2: Portoljunk C++ kódot

- A tárgy oldalán elérhető egy rövid C++ kódrészlet, portoljuk Rust-ra
 - Lehetséges-e 1:1 portolni a kódot?
 - Megéri-e?
 - Mely hibákat találta meg a fordító?
 - Milyen egyéb hibákat tartalmaz a C++ kód?

Küzdelem a fordítóval

- Eleinte a Rust fejlesztés elképzelhető hogy a fordítóval való küzdelemnek érződik
 - Ez a gyakorlattal csökken, a fordító segíteni próbál
 - Az új komponensek folyamatosan javítanak ezen
 - A garanciák részben kárpotlásként szolgálhatnak
 - Hosszú távon megtérül a küzdelem
- Van hogy nem tudunk leírni olyan kódot, ami elméletileg helyes

WOAO: Ez itt az első haladó feladat dia

- A tárgy előadásai mellé ezeken a zöld diákon lesznek haladó feladatok
- Ezek megoldását azoknak ajánljuk, akik már ismerik az előadás anyagát
 - A haladóbb laborokon igyekszünk kitérni arra, hogy milyen megoldásokat volt érdemes használni ezekben
- A feladatok megoldása a ZH-n felhasználható extra pontot ér

W1A1: Készíts Huffman kódolót

- A Huffman kód egy optimális prefix kód, amely veszteségmentesen tömörít
- Tetszőleges könyvtár használható
 - Ajánlott a `clap` használata parancssoros felület készítéséhez
 - Huffman kódolást már megvalósító könyvtárak „becsomagolása” nem elfogadható megoldás
- Elérhető extra pontok: 2 | határidő: 2 hét múlva, előadás vége

Köszönöm a figyelmet!